

Numero matricola: 2169979
 Corso: in presenza canale MZ
 Nome e cognome: Matteo Maddalena
 Gruppo: singolo

Struttura del MVC.....	1
Gestione del profilo utente.....	3
Implementazione delle stanze.....	4
Gestione della simulazione.....	5
Rendering della scena.....	7
Sprite animate.....	8
Audio sample.....	8
Menù di gioco.....	8
Pezzi di puzzle.....	9
Gestione degli eventi.....	9
Vari design pattern usati.....	10
Uso degli stream.....	10

Struttura del MVC

Il progetto usa il pattern architetturale MVC; per questa ragione è stato strutturato in tre package: *code.controller*, *code.model* e *code.view*, per contenere rispettivamente il codice del *Controller*, della *Model* e della *View*.

Di seguito la struttura gerarchica dell'intero progetto:

- code.controller
 - code.controller.event
 - ControllerEvent.java
 - GameResumed.java
 - StopGame.java
 - TerminalMenuRequested.java
 - GameLoop.java
 - JImpossibleMission.java
- code.model
 - code.model.context
 - AttackEnded.java
 - AttackLaunched.java
 - FurnitureSearchEnded.java
 - GameContext.java
 - GameWillEnd.java
 - ModelEvent.java
 - ModelState.java
 - PlayerDied.java
 - PlayerFoundSomething.java
 - PlayerIsSearching.java
 - PlayerOnPlatform.java

- RunnerStartRunning.java
- RunnerStopRunning.java
- StopSimulation.java
- TerminalOpened.java
- code.model.gameobjects
 - code.model.gameobjects.enemy
 - AttackerRobot.java
 - BlackOrb.java
 - Enemy.java
 - EnemyFactory.java
 - JumperRobot.java
 - LaserRobot.java
 - RunnerRobot.java
 - ThrowerRobot.java
 - FixedObject.java
 - Furniture.java
 - GameObject.java
 - GameObjectFactory.java
 - MovingObject.java
 - Platform.java
 - PlatformCluster.java
 - Player.java
 - Terminal.java
- code.model.puzzle
 - PresettedPassword.java
 - PuzzlePiece.java
- code.model.room
 - PresettedRoom.java
 - RoomMap.java
 - Room.java
 - RoomParser.java
- GameWorld.java
- Leaderboard.java
- Point.java
- code.view
 - code.view.audio
 - AudioBank.java
 - AudioData.java
 - AudioPlayer.java
 - AudioUtils.java
 - code.view.images
 - Animation.java
 - ImageUtils.java
 - StaticImage.java
 - code.view.menu
 - code.view.menu.event
 - CloseGame.java
 - GamePanelRequested.java

- InputBoxMenuRequested.java
 - LeaderboardMenuRequested.java
 - MenuEvent.java
 - PlatformResetRequested.java
 - PuzzleMenuClosed.java
 - PuzzleMenuOpened.java
 - PuzzleMenuRequested.java
 - RobotDisableRequested.java
 - SecondaryMenuClosed.java
 - TerminalClosed.java
 - LeaderboardMenu.java
 - Menu.java
 - MenuButton.java
 - PlayerNameMenu.java
 - PuzzleMenu.java
 - TerminalMenu.java
- code.view.sprites
 - AnimatedSprite.java
 - BlackOrbSprite.java
 - FloorSprite.java
 - FurnitureSprite.java
 - JumperRobotSprite.java
 - LaserRobotSprite.java
 - PlatformSprite.java
 - PlayerSprite.java
 - RunnerRobotSprite.java
 - SearchingWindow.java
 - Sprite.java
 - SpriteFactory.java
 - TerminalSprite.java
 - ThrowerRobotSprite.java
 - WallSprite.java
- Renderer.java
- code.event
 - EventDispatcher.java
 - EventListener.java

La *Model* è completamente indipendente da *View* e *Controller*, mentre questi ultimi dipendono mutuamente e dalla *Model*.

La comunicazione fra queste tre componenti avviene tramite l'uso degli eventi.

Gestione del profilo utente

Il gioco tiene soltanto traccia del nome del giocatore e della sua posizione nella classifica globale; quando il giocatore vuole iniziare una partita scrive il suo nickname, e quando questa termina (per la vittoria del giocatore o per la sua morte in gioco) viene salvato nella classifica insieme al punteggio ottenuto, calcolato seguendo questi criteri:

1. se il giocatore non trova nulla dentro un mobile, non gli vengono assegnati punti

2. se il giocatore trova in un mobile una password per il blocco dei robot o per il ripristino delle piattaforme, gli vengono aggiunti 250 punti
3. se il giocatore trova in un mobile un pezzo di puzzle, guadagna 1000 punti
4. la morte del giocatore determina la perdita di 350 punti
5. il punteggio del giocatore non può scendere sotto lo zero

Se il giocatore inserisce un nickname già utilizzato, il gioco aggiornerà il punteggio in classifica associato a quel nickname solo se maggiore del precedente; evitando così di peggiorare il punteggio del vero proprietario nel nome.

La classifica è implementata dalla classe *code.model.Leaderboard*, serializzabile e singleton: il metodo *load()*, se non è già presente un'istanza, permette di caricare la classifica in memoria creando un'istanza della classe; il metodo *store()* permette invece di memorizzare la classifica sul disco, così da non perdere il suo contenuto fra le diverse esecuzioni del gioco; il metodo *addEntry(Entry entry)* permette invece di aggiungere una riga alla classifica (con la entry che è una coppia (nome, punteggio)).

La GUI della classifica è invece implementata dalla classe *code.view.menu.LeaderboardMenu*, che estende *JPanel* e aggiunge a se stessa tutte le componenti necessarie al menù (chiamando *this.add(Component comp)* di *JPanel*). Infine il menù che permette all'utente di inserire il suo nome è implementato dalla classe *code.view.menu.PlayerNameMenu*, che funziona come quella per la classifica.

Funzionalità come avatar, contatori di partite vinte/perse e livello non sono state implementate per scelta progettuale, privilegiando la completezza delle meccaniche di gioco.

Implementazione delle stanze

Le stanze del gioco sono implementate tramite l'uso di diverse classi ed enumerazioni che svolgono ruoli distinti:

1. l'enumerazione *code.model.room.RoomMap* memorizza sotto forma di matrici di interi le 34 mappe del gioco: ogni mappa viene qui rappresentata come una matrice di tiles 25x40. Tutte le tiles hanno la stessa dimensione e un tipo preciso (WALL, FLOOR, ENEMY...)
2. la classe astratta di utilità generale *code.model.room.RoomMapParser* si occupa di tradurre la rappresentazione matriciale di una stanza nella rappresentazione a lista di *GameObject* usata successivamente dalla *Model*
3. l'enumerazione *code.model.room.PresettedRoom* contiene invece tutte le informazioni per creare un'istanza di *Room* per la stanza desiderata (funge da blueprint per le 34 stanze del gioco)
4. infine la classe *code.model.room.Room* modella il concetto di stanza del gioco e contiene la lista dei *GameObject* nella stanza

Gestione della simulazione

La mappa di gioco consiste in 34 stanze distribuite casualmente, garantendo la giocabilità e l'attraversabilità, in una matrice 6x17. Quattro di queste sono usate per le colonne degli ascensori, le restanti invece sono normali stanze dove giocare, affrontare i robot e frugare nei mobili in cerca di qualcosa.

La mappa è implementata dalla classe `code.model.GameWorld`, che non è un singleton perché viene istanziata ogni volta che il giocatore inizia una nuova partita; il costruttore infatti si occupa, tramite il metodo `randomGeneration()`, di creare casualmente la matrice della mappa e quindi anche una nuova istanza della classe.

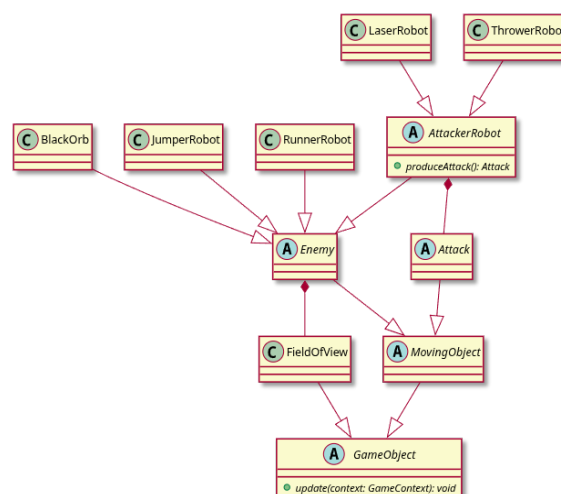
Il giocatore naviga la mappa semplicemente muovendosi nella matrice che la rappresenta.

Ad aspettare il giocatore ci sono cinque tipi di nemici:

1. i runner robot, implementati dalla classe `code.model.gameobjects.enemy.RunnerRobot`, che rincorrono il giocatore non appena entra nel loro campo visivo
2. i thrower robot, implementati dalla classe `code.model.gameobjects.enemy.ThrowerRobot`, che lanciano una sfera verso il giocatore non appena entra nel loro campo visivo
3. i jumper robot, implementati dalla classe `code.model.gameobjects.enemy.JumperRobot`, che iniziano a saltare quando vedono il giocatore
4. i laser robot, implementati dalla classe `code.model.gameobjects.enemy.LaserRobot`, che sferrano un attacco laser appena vedono il nemico
5. le black orb, implementate dalla classe `code.model.gameobjects.enemy.BlackOrb`, che volando inseguono il giocatore

I nemici sono implementati seguendo una gerarchia per le classi ben precisa: la classe `code.model.gameobjects.enemy.Enemy` modella il nemico generico ed estende la classe `code.model.gameobjects.MovingObject` usata per tutto ciò che è in movimento; le classi `RunnerRobot`, `BlackOrb` e `JumperRobot` estendono la classe `Enemy` ed implementano le specifiche logiche di aggiornamento; la classe `code.model.gameobjects.enemy.AttackerRobot` modella invece i robot che possono lanciare degli attacchi: `LaserRobot` e `ThrowerRobot` la estendono e implementano le loro logiche specifiche per l'aggiornamento del robot e la produzione degli attacchi.

Ogni nemico ha inoltre un campo visivo implementato dalla inner class `Enemy.FieldOfView`. I nemici che possono attaccare hanno invece la inner class `AttackerRobot.Attack` che modella l'attacco.

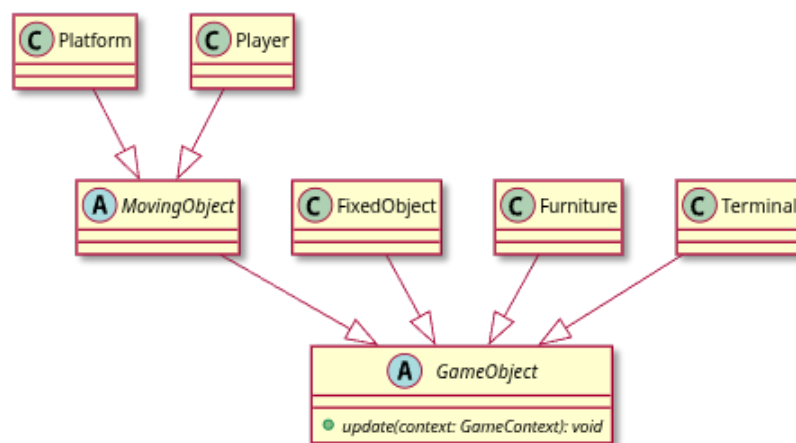


Le classi *Enemy* e *AttackerRobot* sono astratte perché soltanto le classi finali, cioè quelle che modellano i nemici del gioco, devono poter essere istanziate. La classe *AttackerRobot.Attack* è astratta invece perché gli attacchi vengono istanziati usando il meccanismo delle classi anonime.

La logica di aggiornamento dei robot viene implementata sovrascrivendo il metodo *update(GameContext context)* della classe *GameObject*. Gli attacchi vengono invece prodotti sovrascrivendo il metodo *produceAttack()* della classe *AttackerRobot.Attack*.

Ogni oggetto del gioco (i terminali, i robot, il giocatore, le piattaforme, i muri, i pavimenti e i mobili) viene implementato estendendo la classe *GameObject* o una delle sue sottoclassi, come *MovingObject*.

Di seguito viene riportata la struttura delle classi degli oggetti di gioco esclusa quella dei nemici perché già mostrata in precedenza:



La classe *code.model.gameobjects.GameObject* modella tutti gli oggetti del gioco, mentre la classe *MovingObject* soltanto quelli in movimento, per i quali quindi applicare la gravità e il calcolo delle collisioni.

Le classi *code.model.gameobjects.FixedObject*, *code.model.gameobjects.Furniture* e *code.model.gameobjects.Terminal* rappresentano rispettivamente i pavimenti e i muri, i mobili e i terminali, estendono tutte la classe *GameObject* e implementano la loro logica di aggiornamento sovrascrivendo il metodo *update(Game Context context)* ereditato da *GameObject*.

Le classi *code.model.gameobjects.Platform* e *code.model.gameobjects.Player* modellano invece le piattaforme e il giocatore e sono gli unici oggetti di gioco, insieme ai nemici, che si possono muovere.

Ogni stanza viene modellata usando la classe *code.model.room.Room* che al suo interno contiene una lista di *GameObject* che rappresenta gli oggetti di gioco contenuti nella stanza. Tramite questa lista la classe *code.controller.GameLoop*, dalla quale viene creato il thread del game loop, darà vita alla stanza chiamando ripetutamente il metodo *update(Game Context context)* su tutti i *GameObject* della lista.

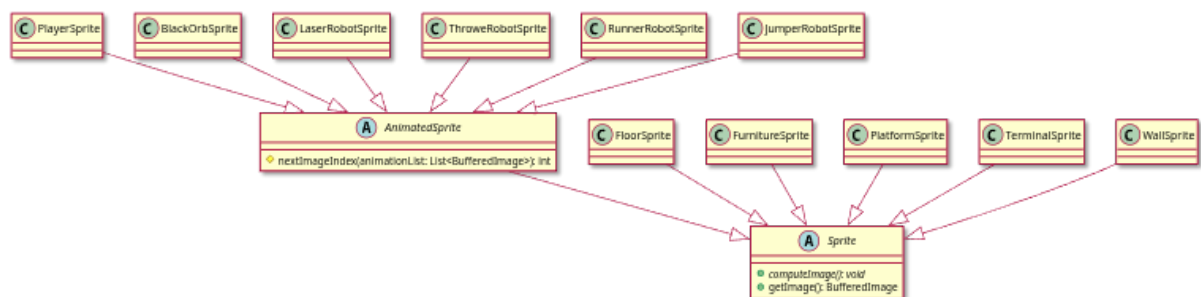
La classe *code.model.context.GameContext* è usata per rappresentare il contesto di gioco: memorizza informazioni come il giocatore che sta giocando, il mondo generato, la classifica

caricata, l'input dell'utente, se i robot sono stati disabilitati, se le piattaforme vanno ripristinate, ed è essenziale per informare le classi della *Model* sullo stato attuale della partita. Per esempio il metodo *update(Game Context context)* della classe *GameObject* prende come parametro un'istanza di *GameContext* permettendo così a tutti gli oggetti del gioco di prendere decisioni opportune (come quella del *Player* di morire quando sta collidendo con un nemico).

Rendering della scena

Utili al rendering della scena sono le classi che implementano le sprite degli oggetti di gioco. Con il termine sprite nel mio progetto non si intende un'immagine statica per un oggetto, bensì un'astrazione che racchiude in sé il *GameObject* associato all'immagine e tutta la logica usata per il calcolo di questa in un determinato istante di tempo.

Di seguito la gerarchia delle sprite:



La classe astratta *view.sprites.Sprite* modella le sprite nel senso dato prima, mentre la classe *view.sprites.AnimatedSprite* è usata per le sprite la cui immagine nel tempo può cambiare. Il metodo astratto *computeImage()* della classe *Sprite* è usato per calcolare l'immagine corrente della sprite e viene sovrascritto dalle classi che estendono *Sprite* in due modi:

1. le classi *FloorSprite*, *FurnitureSprite*, *PlatformSprite*, *TerminalSprite* e *WallSprite* non cambiano l'immagine nel tempo e quindi la loro implementazione del metodo si limita a impostare l'immagine della sprite sempre allo stesso modo
2. la classe *AnimatedSprite* implementa il metodo in modo da cambiare l'immagine della sprite seguendo gli indici per la lista delle animazioni restituiti da *nextImageIndex(List<BufferedImage> animationList)*; le sottoclassi di *AnimatedSprite* sovrascrivono questo metodo cambiando a piacimento il modo in cui scorrere la lista delle animazioni e quindi l'output di *computeImage()*

La classe *view.Renderer* mette tutto insieme generando tramite il metodo *setCurrentSpriteList()* le sprite per la lista degli oggetti della stanza corrente e chiamando successivamente il metodo *computeImage()* su ognuna di queste. Una volta ottenute le immagini da visualizzare nel frame, stampa ogni sprite nella posizione del *GameObject* ad essa associato.

Sprite animate

Le sprite animate, modellate dalla classe *AnimatedSprite*, sono state implementate tramite un sistema di accumulo del tempo passato molto rudimentale: il thread del game loop del

gioco chiama ad ogni iterazione il metodo *updateElapsedTime(double deltaTime)* delle *AnimatedSprite* passandogli il tempo in secondi passato dall'ultima iterazione (il Δt); ogni *AnimatedSprite* allora aggiunge il Δt al tempo passato dall'ultimo aggiornamento dell'animazione (*elapsedTime* += Δt) e non cambia immagine finché *elapsedTime* < *imageDuration* (la durata del singolo frame dell'animazione). Quando la condizione non è più soddisfatta (*elapsedTime* ≥ *imageDuration*), allora vuol dire che il frame corrente è stato visualizzato abbastanza e bisogna passare al successivo. Può succedere che un frame venga visualizzato più del dovuto quando il game loop non riesce a chiamare abbastanza spesso il metodo *updateElapsedTime(double deltaTime)*: se per esempio il metodo viene chiamato una volta al secondo, e quindi ci sarà un aggiornamento dell'*elapsedTime* al secondo, e la durata del frame è di mezzo secondo, ogni frame viene visualizzato per mezzo secondo in più. Per risolvere questo problema, quando è ora di passare al frame successivo l'*elapsedTime* non viene azzerato ma impostato a *elapsedTime* -= *imageDuration*, in modo da garantire la durata voluta dell'intera animazione a costo di fare durare alcuni frame di meno.

Questo metodo garantisce l'indipendenza dell'animazione dalla velocità dell'hardware.

Audio sample

La riproduzione degli audio sample viene implementata con diverse classi ed enumerazioni:

1. la classe *code.view.audio.AudioData* contiene i dati grezzi di una traccia audio (cioè di tipo *byte[]*) e il suo formato
2. la classe astratta di utilità generale *code.view.audio.AudioUtils* definisce invece vari metodi di utilità generale per lavorare sugli audio sample: il metodo *loadAudioSample(String pathname)* permette di caricare una traccia audio dal disco, mentre il metodo *toClip(AudioData audioSample)* permette di convertire una traccia audio in una risorsa utilizzabile per la sua riproduzione
3. l'enumerazione *code.view.audio.AudioBank* memorizza invece tutte le tracce audio usate dal gioco
4. infine la classe *code.view.audio.AudioPlayer* si occupa di iniziare e gestire la riproduzione delle tracce audio: permette di avviare la musica di sottofondo e di far partire e fermare suoni quando il giocatore muore, i nemici attaccano o in altre circostanze

Menù di gioco

Ogni menù del gioco è implementato da un'apposita classe:

1. la classe *code.view.menu.LeaderboardMenu* per il menù della classifica
2. la classe *code.view.menu.Menu* per il menù principale
3. la classe *code.view.menu.PlayerNameMenu* per il menù dove l'utente inserisce il suo nickname
4. la classe *code.view.menu.PuzzleMenu* per il menù che permette all'utente di comporre i pezzi di puzzle
5. la classe *code.view.menu.TerminalMenu* per il menù dei terminali dove l'utente può disattivare i robot e ripristinare le posizioni delle piattaforme

Tutti i menù estendono *JPanel* e impostano i loro componenti quando vengono costruiti chiamando su sé stessi i metodi ereditati dalla superclasse (per esempio *this.add(Component comp)*); ogni menù racchiude quindi in sé tutta le istruzioni per costruirlo.

Ad occuparsi dell'inserimento dei menù nel frame della finestra di gioco e del passaggio da un menù all'altro quando necessario è il *Controller* che, registrando i listener per gli eventi generati dai menù, gestisce tutto ciò. Per esempio:

```
startButton.addActionListener(
    e -> EventDispatcher.notify(new InputBoxMenuRequested())
); // fatto dalla view nella classe Menu
...
EventDispatcher.subscribeAsStatic(
    InputBoxMenuRequested.class,
    x -> layout.show(rootPanel, PLAYER_MENU_ID)
); //fatto dal controller nella classe JImpossibleMission
```

in questo pezzo di codice il menù *Menu* notifica l'evento *InputBoxMenuRequested* quando il bottone per iniziare la partita viene cliccato; il controller, nella classe *JImpossibleMission*, registra il listener per quell'evento che a sua volta si occupa di mostrare il menù *PlayerNameMenu*.

La *View* racchiude quindi in sé l'aspetto dei menù, mentre il *Controller* gestisce la logica di navigazione fra i menù.

Pezzi di puzzle

Il gioco modifica la meccanica dei pezzi di puzzle originariamente presente in *Impossible Mission*: all'avvio della partita, viene scelta una password casuale, ogni lettera della password viene poi scomposta in quattro pezzi, e infine i pezzi così ottenuti vengono distribuiti nei mobili della mappa; la sfida che deve affrontare il giocatore è quella di trovare i pezzi delle lettere e ricomporre la password (che è nota).

Il menù che permette di ricomporre i pezzi di puzzle è il menù *PuzzleMenu*.

Il rompicapo è implementato tramite:

1. l'enumerazione *code.model.puzzle.PresettedPassword*, che contiene tutte le nove password usate dal gioco
2. l'enumerazione *code.model.puzzle.PuzzlePiece*, che contiene tutti i pezzi di lettere che compongono una password

La classe *GameWorld* durante la generazione della mappa, sceglie una password casuale, chiama il metodo *getPieces(String password)* di *PuzzlePiece* per ottenere la lista dei pezzi di lettere che compongono la password scelta, e infine li distribuisce nei mobili della mappa generata.

Il tentativo dell'utente di comporre la password viene controllato dal metodo *checkAttempt(PuzzlePiece[][] attempt)* che, dato il tentativo sotto forma di lista di matrici 2x2 (ogni matrice rappresenta una lettera e ogni cella un pezzo di lettera), restituisce *true* se

e solo se il giocatore ha composto correttamente la password. Il controllo del tentativo è particolarmente semplice grazie alle informazioni memorizzate nei pezzi di puzzle: dato che ogni pezzo di puzzle memorizza la sua posizione (*UPPER_RIGHT*, *UPPER_LEFT*, *BOTTOM_RIGHT*, *BOTTOM_LEFT*) e il carattere da cui deriva, allora il metodo deve soltanto controllare se vale la condizione $M[0][0].getPosition() = UPPER_LEFT \wedge M[0][1].getPosition() = UPPER_RIGHT \wedge M[1][0].getPosition() = BOTTOM_LEFT \wedge M[1][1].getPosition() = BOTTOM_RIGHT \wedge M[0][0].getLetter() = M[0][1].getLetter() = M[1][0].getLetter() = M[1][1].getLetter() = Password[i] \forall i \in \{0, \dots, Password.length - 1\}$

Gestione degli eventi

Piuttosto che utilizzare il classico design pattern Observer/Observable, il gioco implementa la comunicazione fra le classi tramite l'uso di un HUB centrale che riceve e instrada gli eventi (l'Event Bus pattern). Il nodo centrale è implementato dalla classe astratta *code.event.EventDispatcher* che con il metodo *subscribe(Class<T> type, EventListener<T> listener)* permette di registrare uno o più handler per ogni evento, mentre con il metodo *notify(T event)* permette di notificare un evento.

code.event.EventListener è invece un'interfaccia funzionale che stabilisce quale metodo i listener debbano implementare per poter gestire correttamente gli eventi; nonostante la sua importanza nessuna classe la implementa esplicitamente perché tutti i listener sono stati istanziati con il meccanismo delle classi anonime.

Ogni evento è inoltre identificato e tipizzato da una classe, e per registrare un listener per un dato evento bisogna specificare la classe dell'evento piuttosto che una sua istanza (fare cioè uso della reflection). Di conseguenza il metodo *notify(T event)*, nonostante prenda l'istanza dell'evento, considera la classe dell'evento.

Un esempio di utilizzo:

```
EventDispatcher.subscribe(AttackLaunched.class, x -> { ... })
...
EventDispatcher.notify(new AttackLaunched())
```

È stato scelto l'Event Bus pattern perché più scalabile rispetto all'Observer/Observable: le classi stavano diventando tante e il codice illeggibile.

Inoltre, gli eventi sono stati utilizzati per permettere la comunicazione fra *Model*, *View* e *Controller* senza creare dipendenze e anche per sfruttare i vantaggi e le comodità del paradigma ad eventi.

Vari design pattern usati

Per lo sviluppo del gioco sono stati usati quattro tipi di design pattern, vediamo:

1. **Template** pattern: usato dalla classe *AnimatedSprite* con il metodo *computeImage()* che implementa lo scheletro dell'algoritmo per aggiornare le animazioni e usa *nextImageIndex(List<BufferedImage> animationList)* che delega alle sottoclassi come scansionare la lista della animazioni
2. **Factory** pattern: sono state usate due factory: la classe *code.model.gameobjects.GameObjectFactory* che implementa una factory di *GameObject* e la classe *code.view.sprites.SpriteFactory* che implementa una factory di *Sprite*
3. **Event Bus** pattern: usato al posto dell'Observer/Observable per la gestione degli eventi

4. **Singleton** pattern: usato per la classe *Leaderboard* e *AudioSample*, entrambe classi istanziabili solamente una volta

Uso degli stream

Gli stream sono stati usati moltissimo durante lo sviluppo del progetto in tutti quei casi dove è emersa la necessità di elaborare un flusso di dati in maniera omogenea. Di seguito vengono riportati vari pezzi di codice dove se ne fa uso.

```
renderer.getCurrentSpritesList().stream().filter(s ->
    (!context.isRobotsDisabled() && s instanceof AnimatedSprite) ||
    s instanceof PlayerSprite
).forEach(s -> ((AnimatedSprite)s).updateElapsedTime(dt));
```

Usato per aggiornare l'*elapsedTime* delle *AnimatedSprite* ad ogni iterazione del game loop: si prende la lista delle sprite correnti, la si trasforma in uno stream, si eliminano gli elementi che non soddisfano la condizione e si chiama su ogni elemento il metodo *updateElapsedTime(double deltaTime)*.

```
List<Furniture> allFurnitures =
Arrays.stream(PresettedRoom.values()).flatMap(r ->
    r.getFurnitures().stream()
).collect(Collectors.toList())
```

Usato per ottenere la lista di tutti i mobili nella mappa di gioco: si converte in stream l'array delle *PresettedRoom*, si chiama su *PresettedRoom* il metodo *getFurnitures()*, si converte ogni lista così ottenuta in uno stream, si rende lo stream di stream un solo stream e infine si converte lo stream in una lista.

```
List<Point> pointToUse = IntStream.range(0, cols).filter(x ->
    x % 2 == 0
).mapToObj(x ->
    IntStream.range(0, rows).mapToObj(y ->
        new Point(x, y))
).flatMap(p -> p).collect(Collectors.toList());
```

Usato per ottenere la lista di punti da usare nella generazione casuale del mondo: crea uno stream di valore interi da 0 a *cols - 1*, elimina i valori dispari, converte i numeri in uno stream di punti, rende lo stream di stream di punti uno stream di punti e lo converte in una lista.

```
private static final PresettedRoom[] rightRoom = Arrays.stream(values())
    .filter(r ->
        r.exitLayout == Room.ExitLayout.ONRIGHT && !r.isElevator
    ).toArray(PresettedRoom[]::new);
```

Usato nella classe *PresettedRoom* per ottenere tutte le stanze con l'uscita a destra: crea uno stream dall'array di valori dell'enumerazione, seleziona solamente le istanze enumerative con l'uscita a destra e che non sono ascensori e infine converte lo stream in un array.

```
upperPlatform = platforms.stream().min(
    Comparator.comparingDouble(p -> p.getPosition().getY())
).get();
```

Usato nella classe *PlatformCluster* per trovare la piattaforma più in alto in un cluster di piattaforme: trasforma la lista di piattaforme in uno stream, cerca il minimo usando il principio stabilito dal *Comparator* e infine prende il minimo.

```
return floorList.stream().anyMatch(f ->
    f.getPosition().getY() < upperPlatformY &&
    (f.getPosition().getX() == upperPlatformX + width ||
    f.getPosition().getX() + f.getWidth() == upperPlatformX)
);
```

Usato nella classe *PlatformCluster* per capire se il cluster può continuare a salire: converte la lista di pavimenti nella stanza in uno stream e controlla se è presente almeno un elemento che soddisfa la condizione. Se c'è restituisce true, altrimenti false

```
return room.getPlatformList().stream().collect(
    Collectors.groupingBy(p -> p.getPosition().getX())
).values().stream().map(pl -> new PlatformCluster(pl)).toList();
```

Usato nella classe *PlatformCluster* per creare la lista di cluster di platform nella stanza: converte la lista di platform nella stanza in uno stream, raggruppa tutte le platform con la stessa coordinata x creando così una mappa x -> lista di platform, converte la lista dei valori della mappa in uno stream, mappa ogni lista di platform in un oggetto di *PlatformCluster* e infine converte tutto in una lista.